

SETUP GUIDE

SpaceLift

Step-by-Step Deployment Guide

September 2026

Contents

1. What You Will Need — Overview
2. Step 1: Install Prerequisites on Your Computer
3. Step 2: Set Up the Database (Supabase)
4. Step 3: Run Database Migrations
5. Step 4: Set Up File Storage Buckets
6. Step 5: Deploy the AI Edge Function
7. Step 6: Get and Store AI API Keys
8. Step 7: Create Your .env File
9. Step 8: Build and Run the Website
10. Step 9: Deploy to a Live Server
11. Step 10: Verify Everything Works
12. Quick Checklist
13. Troubleshooting Common Issues

1. What You Will Need — Overview

SpaceLift is a web application built with React. It uses a cloud database to store user accounts, projects, and shopping lists, and it calls an AI service to generate room designs. Here is what you need, in plain terms:

Component	What It Does	Cost	Required?
Node.js	Runs the build tools that turn the code into a live website	Free	Required
Supabase (cloud)	Hosts the database, user logins, file storage, and the AI function	Free tier	Required
OpenAI API Key	Powers the AI room design image generation and chat assistant	Paid	Optional*
A hosting service	Puts your website on the internet so anyone can visit it	Free tier	Required

* **About the OpenAI key:** SpaceLift will run without an AI key, but the AI features will be limited. The chat assistant will use built-in fallback responses instead of real AI, and image generation will show the user's original photo instead of an AI-designed version. For the full experience, an OpenAI API key is needed.

Here is the big picture of how the pieces fit together:

1. **Your computer** — where you install tools, edit code, and build the site
2. **Supabase (cloud)** — where the database, user accounts, photo storage, and AI function live
3. **OpenAI (cloud)** — called by the AI function to generate images and chat responses
4. **A hosting service (cloud)** — where the finished website files are published so visitors can access them

Good news: Your Supabase project is already provisioned. The database URL and anon key are already in your `.env` file. You do not need to create a new Supabase project from scratch — you just need to make sure the migrations have been applied and the edge function is deployed (Steps 3-5 below).

2. Step 1: Install Prerequisites on Your Computer

A Install Node.js (version 18 or higher)

Node.js is the engine that runs the build tools. Think of it as the power tool you need to assemble the website.

- Go to **<https://nodejs.org>**
- Download the "LTS" (Long Term Support) version — it is the most stable
- Run the installer and follow the prompts (default settings are fine)
- To verify it installed, open a terminal (Command Prompt on Windows, Terminal on Mac) and type: `node --version` — you should see a version number like `v20.17.0`

B Get the Project Files

If you are reading this, you likely already have the project files. If not, copy the entire project folder to your computer. Make sure the folder contains files like `package.json`, `vite.config.ts`, and the `src/` folder.

C Install the Project's Dependencies

The project relies on several free code libraries (like React and Tailwind CSS). You need to download them once.

1. Open a terminal
2. Navigate to the project folder (e.g., `cd /path/to/project`)
3. Run: `npm install`
4. Wait — this downloads everything the project needs. It may take a minute or two.

What is happening: The `package.json` file lists all the libraries the project uses. The `npm install` command reads that list and downloads them into a folder called `node_modules`. You never need to touch that folder — it is generated automatically.

3. Step 2: Set Up the Database (Supabase)

Supabase is a free cloud service that provides three things SpaceLift needs:

- **A database** — stores user accounts, projects, design versions, shopping list items, and chat messages
- **File storage** — stores the uploaded room photos and AI-generated design images
- **Edge functions** — small server-side programs that run the AI logic securely

A Access Your Supabase Project

Your Supabase project is already set up. To access it:

1. Go to **<https://supabase.com>** and sign in
2. Click on your project (the URL in your `.env` file tells you which one — the part before `.supabase.co`)
3. You should see the project dashboard with tabs for Table Editor, SQL Editor, Storage, Edge Functions, etc.

B Find Your Project Credentials

You need two pieces of information from Supabase. They are already in your `.env` file, but here is where to find them if you ever need them:

1. In the Supabase dashboard, go to **Settings** (gear icon, bottom left)
2. Click **API**
3. You will see:
 - **Project URL** — looks like `https://yourproject.supabase.co`
 - **anon public key** — a long string starting with `eyJ...`
 - **service_role secret key** — a long string (keep this secret! More on this in Step 6)

Important: The **anon key** is safe to include in your website code — it only allows actions that your database rules explicitly permit. The **service_role key** is a master key that can do anything — never put it in your website code or share it publicly. It only goes in the edge function secrets (Step 5).

4. Step 3: Run Database Migrations

A "migration" is a script that creates or updates your database tables. SpaceLift has 5 migration files in the `supabase/migrations/` folder. They create the following database tables:

Table	What It Stores
<code>projects</code>	Each design project (room type, budget, mood, lease rules, etc.)
<code>design_versions</code>	Saved versions of a design (each regeneration creates a new version)
<code>design_items</code>	Individual shopping list items within a design version
<code>chat_messages</code>	Chat conversation between the user and the AI assistant
<code>move_steps</code>	Step-by-step furniture rearrangement instructions for \$0 budget plans

Each table also has "row-level security" rules that ensure users can only see and modify their own data.

A How to Apply Migrations

If your project was set up through Bolt, the migrations may have already been applied. To check and apply them:

1. In the Supabase dashboard, go to the **SQL Editor**
2. Open each migration file in the `supabase/migrations/` folder, copy its contents, and paste it into the SQL Editor
3. Click **Run** to execute it
4. Repeat for each file, in order (by date in the filename):
 - `20260926132723_create_spacelift_schema.sql` — creates all 5 tables, security rules, and indexes
 - `20260926133745_fix_function_search_path.sql` — fixes a security setting on a database function
 - `20260926152444_add_room_photos_storage.sql` — creates the room photos storage bucket
 - `20260926155946_add_design_image_storage.sql` — creates the design images storage bucket

- `20260926163539_make_storage_buckets_public.sql` — makes those buckets publicly readable

B Verify the Tables Were Created

1. In the Supabase dashboard, go to **Table Editor**
2. You should see 5 tables: `projects`, `design_versions`, `design_items`, `chat_messages`, `move_steps`
3. Click any table to verify it has columns (if it is empty, that is normal — no data has been added yet)

Re-running migrations is safe: Each migration uses `IF NOT EXISTS` and `DROP POLICY IF EXISTS` guards, so running them again will not cause errors or duplicate data.

5. Step 4: Set Up File Storage Buckets

SpaceLift needs two "buckets" (think of them as folders) in Supabase Storage to hold uploaded files:

Bucket Name	What It Holds	Who Can Read	Who Can Write
room-photos	Photos users upload of their rooms	Public (anyone with the link)	Authenticated users only
design-images	AI-generated design images	Public (anyone with the link)	Authenticated users only (via edge function)

A Verify the Buckets Exist

1. In the Supabase dashboard, go to **Storage**
2. You should see two buckets: `room-photos` and `design-images`
3. If they are missing, run the storage migration files (Step 3 covers this)

B Verify the Buckets Are Public

1. In Storage, click on each bucket
2. Click the settings/gear icon or the "Public" toggle
3. Make sure "Public bucket" is turned ON for both
4. This allows the website to display images without requiring a login

6. Step 5: Deploy the AI Edge Function

SpaceLift has one edge function called `ai-design`. It is a small program that lives on Supabase's servers and handles three things:

- **Generating AI room design images** — takes the user's photo and settings, calls OpenAI, and saves the result
- **Generating lighting preview images** — creates an evening-lit version of the room during the intake flow
- **Chat responses** — processes user messages and returns AI-powered swap/redesign instructions

A Deploy the Function

The edge function code is in `supabase/functions/ai-design/index.ts`. To deploy it through Bolt, the function is deployed automatically using the Supabase deploy tool. If you are setting up manually:

1. In the Supabase dashboard, go to **Edge Functions**
2. Click **Create a new function** (or "Deploy")
3. Name it `ai-design`
4. Paste the contents of `supabase/functions/ai-design/index.ts`
5. Click **Deploy**

The function must be named exactly `ai-design`. The website calls it at the URL `https://yourproject.supabase.co/functions/v1/ai-design`. If the name is different, the AI features will not work.

B Verify the Function's Config

The file `supabase/config.toml` must contain this section:

```
[functions.ai-design]
verify_jwt = true
entrypoint = "./functions/ai-design/index.ts"
```

The `verify_jwt = true` setting means the function requires a valid user login token before it will respond. This is a security measure — it prevents unauthorized calls to the AI service.

7. Step 6: Get and Store AI API Keys

The edge function calls OpenAI's API to generate images and chat responses. You need an OpenAI API key for the full AI experience.

A Get an OpenAI API Key

1. Go to **<https://platform.openai.com>** and sign up or sign in
2. Add a payment method (OpenAI charges per use — image generation costs a few cents per image, chat costs fractions of a cent per message)
3. Go to **API Keys** section
4. Click **Create new secret key**
5. Name it "SpaceLift" so you remember what it is for
6. Copy the key immediately — it starts with `sk-` and you will not be able to see it again after closing the window

Treat this key like a password. Anyone who has it can make API calls billed to your account. Never put it in code that is visible to the public (like your website's frontend). It only goes in the Supabase edge function secrets.

B Store the Key in Supabase Edge Function Secrets

The edge function reads its keys from Supabase's secret storage — not from your `.env` file. This keeps the key on the server, where visitors cannot see it.

1. In the Supabase dashboard, go to **Edge Functions**
2. Click on the `ai-design` function
3. Click **Secrets** (or "Environment Variables")
4. Add the following secrets:

Secret Name	Value	Required?
OPENAI_API_KEY	Your OpenAI key (starts with <code>sk-</code>)	Optional (needed for full AI)
SUPABASE_URL	Your project URL (<code>https://yourproject.supabase.co</code>)	Required
SUPABASE_SERVICE_ROLE_KEY	Your <code>service_role</code> key from Settings > API	Required

SUPABASE_ANON_KEY

Your anon public key from Settings > API

Required

Why three Supabase keys? The edge function needs to verify the user's identity (anon key + user's token), then perform privileged operations like saving images to storage (service_role key). The URL tells it where to connect.

c **Alternative: Anthropic API Key**

The edge function also supports Anthropic (Claude) as a fallback for the chat assistant. If you prefer to use Anthropic instead of OpenAI for chat:

1. Go to **<https://console.anthropic.com>** and sign up
2. Create an API key
3. Add it as a secret called ANTHROPIC_API_KEY in the Supabase edge function secrets

Note: If both OPENAI_API_KEY and ANTHROPIC_API_KEY are set, OpenAI is used for chat. Image generation always requires OpenAI (it uses DALL-E). Anthropic only powers the chat assistant.

8. Step 7: Create Your .env File

The `.env` file tells your website how to connect to Supabase. It lives in the root of your project folder. Your project already has one — here is what it should contain:

```
VITE_SUPABASE_URL=https://yourproject.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIs...
```

What each value does

- **VITE_SUPABASE_URL** — The address of your Supabase project. The website uses this to connect to the database and storage.
- **VITE_SUPABASE_ANON_KEY** — A public key that identifies your project. It is safe to include in the website code because the database's row-level security rules prevent unauthorized access — the key alone cannot do anything the rules do not allow.

Important rules about the .env file:

- Never put the **service_role key** or **OpenAI API key** in this file. Those are server-only secrets that go in Supabase edge function secrets (Step 6).
- The `.gitignore` file already excludes `.env` from version control, so it will not be accidentally shared if you use Git.
- If you deploy to a hosting service (Step 9), you will need to enter these same values as environment variables in the hosting service's settings.
- Any variable starting with `VITE_` gets baked into the website code at build time. Variables without the `VITE_` prefix are invisible to the website.

If You Are Setting Up on a New Supabase Project

If you are pointing the website at a different Supabase project than the one it was built with:

1. Replace the URL in `VITE_SUPABASE_URL` with the new project's URL
2. Replace the key in `VITE_SUPABASE_ANON_KEY` with the new project's anon key
3. Run the migrations (Step 3) on the new project
4. Set up the storage buckets (Step 4) on the new project
5. Deploy the edge function (Step 5) to the new project
6. Add the edge function secrets (Step 6) to the new project

9. Step 8: Build and Run the Website

A Run It Locally (for testing)

To see the website on your own computer before publishing it:

1. Open a terminal in the project folder
2. Run: `npm run dev`
3. Open a browser and go to **`http://localhost:5173`**
4. You should see the SpaceLift landing page

Local testing does not require AI keys. The site will work — you can create accounts, start projects, and go through the intake flow. AI image generation and chat will use fallback responses unless the OpenAI key is configured in the edge function secrets.

B Build the Production Version

When you are ready to publish, you need to "build" the site. This optimizes the code, minifies files, and creates a `dist/` folder with everything ready to upload.

1. In the terminal, run: `npm run build`
2. Wait for it to finish — you should see a success message showing the output file sizes
3. The `dist/` folder now contains your website as static files (HTML, CSS, JavaScript, and images)

The `.env` values are baked in at build time. If you change the Supabase URL or anon key, you must rebuild (`npm run build`) for the changes to take effect.

C Preview the Production Build Locally

1. Run: `npm run preview`
2. Open a browser to the URL shown (usually **`http://localhost:4173`**)
3. This runs the exact same files that will be deployed — a good final check before publishing

10. Step 9: Deploy to a Live Server

The build step creates static files in the `dist/` folder. You just need to upload those files to a hosting service. Here are three good free options:

A Option 1: Netlify (Easiest)

1. Go to <https://netlify.com> and sign up (free)
2. Click **Add new site > Deploy manually**
3. Drag your `dist/` folder onto the upload area
4. Netlify gives you a live URL immediately (like <https://spacelift.netlify.app>)
5. **Important:** Go to Site Settings > Build & Deploy > Environment Variables and add:
 - `VITE_SUPABASE_URL` = your Supabase URL
 - `VITE_SUPABASE_ANON_KEY` = your anon key
6. Redeploy after adding the environment variables (drag the `dist/` folder again, or better: connect your Git repo so it auto-builds)

For SPA routing: SpaceLift is a single-page app. Add a `_redirects` file in the `public/` folder with the content `/* /index.html 200` so that all routes load the app correctly. Your project already includes this file.

B Option 2: Vercel

1. Go to <https://vercel.com> and sign up (free)
2. Click **Add New > Project**
3. Import your Git repository, or use the Vercel CLI: `npx vercel`
4. In the project settings, add the same two environment variables (`VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`)
5. Set the build command to `npm run build` and the output directory to `dist`
6. Click **Deploy**

C Option 3: Cloudflare Pages

1. Go to <https://pages.cloudflare.com> and sign up (free)
2. Click **Create a project > Upload assets** (for manual) or connect your Git repo
3. If uploading manually, upload the contents of your `dist/` folder
4. If connecting Git, set build command to `npm run build` and output directory to `dist`
5. Add the environment variables in Settings > Environment Variables

6. Click **Deploy**

D Option 4: Any Static File Host

Since the build produces plain HTML/CSS/JS files, you can host them anywhere that serves static files: AWS S3, Google Cloud Storage, GitHub Pages, or even a traditional web server like Nginx or Apache. Just upload the contents of the `dist/` folder. Make sure the server is configured to serve `index.html` for all routes (SPA fallback).

Do NOT upload the entire project folder. Only the contents of `dist/` should go on the hosting service. The `src/` folder, `node_modules/`, and `.env` should never be on the public server.

11. Step 10: Verify Everything Works

Once deployed, go through this checklist to make sure every feature is working:

1 **Landing Page**

Visit your live URL. You should see the SpaceLift landing page with the hero section, animated room preview, and feature cards.

2 **Account Creation**

Click "Design my space" or "I have an account." Try creating an account with an email and password. You should be signed in and redirected to the dashboard.

If this fails, check that the Supabase URL and anon key in your environment variables are correct.

3 **Photo Upload**

Start a new project, upload a room photo. If the upload fails, check that the room-photos storage bucket exists and is set to public.

4 **AI Image Generation**

Complete the intake flow and click "Generate my design." If you have an OpenAI API key configured in the edge function secrets, you should see an AI-generated room design image. If not, you will see your original photo (fallback mode).

5 **Chat Assistant**

Open the chat panel in the workspace and send a message like "Make it more festive." With an OpenAI key, you will get an AI response with a redesign action. Without a key, you will get a built-in fallback response.

6 **Shopping List**

Verify the shopping list shows items with prices, store names, and damage badges. Try locking an item, marking one as "I own this," and opening the swap drawer.

7 **Version History**

Make a change (via chat or editing parameters), then check the version history panel. You should see a new version appear. Try restoring an older version.

12. Quick Checklist

Print this page and check off each item as you go:

#	Task	Done?
1	Node.js installed (version 18+)	___
2	Project files downloaded	___
3	<code>npm install</code> completed	___
4	Supabase project accessible	___
5	Database migrations applied (5 SQL files run)	___
6	5 tables visible in Table Editor	___
7	Storage buckets <code>room-photos</code> and <code>design-images</code> exist	___
8	Both storage buckets set to public	___
9	Edge function <code>ai-design</code> deployed	___
10	OpenAI API key obtained (optional but recommended)	___
11	Edge function secrets set (OPENAI_API_KEY, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY)	___
12	<code>.env</code> file has VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY	___
13	<code>npm run dev</code> works locally	___
14	<code>npm run build</code> completes without errors	___
15	<code>dist/</code> folder uploaded to hosting service	___
16	Environment variables set on hosting service	___
17	Live site loads and shows landing page	___
18	Account creation works	___
19	Photo upload works	___

20	AI design generation works (or fallback shows)	_____
21	Chat assistant responds	_____
22	Shopping list displays items	_____
23	Version history saves and restores	_____

13. Troubleshooting Common Issues

"Missing Supabase environment variables" error

Your `.env` file is missing or the values are wrong. Make sure it is in the root of the project (not inside `src/`) and contains both `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`. After editing, restart `npm run dev` or rebuild.

Sign-up or sign-in fails

Check that your Supabase project has email authentication enabled (it is on by default). In the Supabase dashboard, go to Authentication > Providers and make sure "Email" is enabled. Email confirmation should be OFF (the app auto-signs-in after signup).

Photo upload fails

- Check that the `room-photos` bucket exists in Supabase Storage
- Check that the bucket is public
- Check that the user is signed in (uploads require authentication)
- Check that the photo is under 5MB and is a supported format

AI design image does not generate

- Check that `OPENAI_API_KEY` is set in the edge function secrets (not in `.env`)
- Check that the edge function is deployed (go to Edge Functions in Supabase and verify `ai-design` is listed)
- Check the edge function logs in Supabase for error messages
- Verify your OpenAI account has billing set up and the key is valid
- If no key is set, the app will show the user's original photo as a fallback — this is expected behavior, not a bug

Chat assistant gives generic responses

This happens when no AI key is configured. The app uses built-in fallback responses that match common requests (swaps, budget, evening view) but cannot handle complex or unexpected questions. Add an OpenAI or Anthropic key to the edge function secrets for full AI responses.

Blank page after deploying

- Make sure you uploaded the contents of `dist/`, not the entire project folder

- Make sure the hosting service has the environment variables set (`VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`)
- If you set environment variables after the first deploy, you need to rebuild and redeploy — the values are baked into the build
- Check that the `_redirects` file is in the deployed files (it should be in `dist/`)

"npm run build" fails

- Make sure you ran `npm install` first
- Check that Node.js version is 18 or higher (`node --version`)
- Delete the `node_modules` folder and `package-lock.json`, then run `npm install` again
- Read the error message — it usually points to the specific file and problem

Edge function returns 401 Unauthorized

This means the user's login token is not being passed correctly. Make sure `verify_jwt = true` in `supabase/config.toml` and that the user is signed in before triggering AI features. The website handles this automatically — if you see 401 errors, the user's session may have expired (sign out and sign back in).

Summary

To run SpaceLift on a new server, you need: (1) Node.js on your computer, (2) a Supabase project with the database migrations applied, storage buckets created, and the AI edge function deployed, (3) an OpenAI API key stored as a Supabase edge function secret for full AI features, (4) a `.env` file with the Supabase URL and anon key, and (5) a static hosting service to upload the built `dist/` folder. The entire backend lives on Supabase — there is no separate server to maintain. The website itself is just static files that talk to Supabase directly.